Symbiosis Institute of Technology, Pune

Faculty of Engineering
CSE- Academic Year 2026-27
Java Programming – Lab Batch 2024-28

Lab Assignment No:- 26	
Name of Student	Abishek Yadav
PRN No.	25070122014
Batch	A1
Class	CSE
Academic Year & Semester	2026-2027/3RD
Date of Submission	15/10/2026
Title of Assignment:	Write a Java program to create and execute multiple threads using the Runnable interface.
Theory:	The Runnable interface provides an alternative approach to creating threads in Java. A class implementing Runnable defines the run() method that contains the code executed by the thread. The Runnable object is passed to a Thread object, which starts execution using the start() method. Unlike extending the Thread class, implementing Runnable allows a class to inherit from another superclass, making the design more flexible. Runnable separates task definition from thread execution, resulting in better code organization and reusability. It is widely used in multithreaded enterprise applications and thread pool implementations.
Practice Questions	1. Create a program using Runnable to run two tasks simultaneously.

CODE:

```java
class Task1 implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Task 1: " + i);
            try {
                Thread.sleep(300);
            } catch (InterruptedException e) {
                System.out.println("Task 1 interrupted.");
            }
        }
    }
}

class Task2 implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Task 2: " + i);
            try {
                Thread.sleep(300);
            } catch (InterruptedException e) {
                System.out.println("Task 2 interrupted.");
            }
        }
    }
}

public class exp26 {
    public static void main(String[] args) {
        Runnable task1 = new Task1();
        Runnable task2 = new Task2();

        Thread t1 = new Thread(task1);
        Thread t2 = new Thread(task2);

        t1.setName("Thread 1");
        t2.setName("Thread 2");

        System.out.println("Two tasks started.");

        t1.start();
        t2.start();
```

```
        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }

        System.out.println("Both tasks completed.");
    }
}
```

OUTPUT:

```
C:\Users\Bina Computer\OneDrive\Documents\Java>javac exp26.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\Bina Computer\OneDrive\Documents\Java>java exp26
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Two tasks started.
Task 2: 1
Task 1: 1
Task 2: 2
Task 1: 2
Task 2: 3
Task 1: 3
Task 2: 4
Task 1: 4
Task 2: 5
Task 1: 5
Both tasks completed.

C:\Users\Bina Computer\OneDrive\Documents\Java>
```

QUESTION

Create a Java program using the Runnable interface where two tasks run simultaneously: one thread performs deposit operations, and another performs withdrawal operations, both displaying results concurrently.

CODE:

```
class BankAccount {
    private int balance = 1000;

    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println(
            Thread.currentThread().getName()
            + " deposited Rs." + amount
            + " | Balance = Rs." + balance
        );
    }

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println(
                Thread.currentThread().getName()
                + " withdrew Rs." + amount
                + " | Balance = Rs." + balance
            );
        } else {
            System.out.println(
                Thread.currentThread().getName()
                + " tried to withdraw Rs." + amount
                + " | Insufficient Balance"
            );
        }
    }

    int getBalance() {
        return balance;
    }
}

class DepositTask implements Runnable {
    private BankAccount account;

    DepositTask(BankAccount account) {
        this.account = account;
    }

    public void run() {
        int[] deposits = {500, 700, 300};
        for (int amount : deposits) {
            account.deposit(amount);
            try {
                Thread.sleep(300);
            } catch (InterruptedException e) {
```

```
                System.out.println("Deposit thread interrupted.");
            }
        }
    }
}

class WithdrawalTask implements Runnable {
    private BankAccount account;

    WithdrawalTask(BankAccount account) {
        this.account = account;
    }

    public void run() {
        int[] withdrawals = {400, 600, 500};
        for (int amount : withdrawals) {
            account.withdraw(amount);
            try {
                Thread.sleep(300);
            } catch (InterruptedException e) {
                System.out.println("Withdrawal thread
interrupted.");
            }
        }
    }
}

public class exp26_2 {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        Runnable depositTask = new DepositTask(account);
        Runnable withdrawalTask = new WithdrawalTask(account);

        Thread depositThread = new Thread(depositTask);
        Thread withdrawalThread = new Thread(withdrawalTask);

        depositThread.setName("Deposit Thread");
        withdrawalThread.setName("Withdrawal Thread");

        System.out.println("Bank transaction started.");
        System.out.println("Initial Balance = Rs." +
account.getBalance());

        depositThread.start();
        withdrawalThread.start();

        try {
            depositThread.join();
            withdrawalThread.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }

        System.out.println("Final Balance = Rs." +
account.getBalance());
        System.out.println("Bank transaction completed.");
    }
}
```

OUTPUT:

```
C:\Users\Bina Computer\OneDrive\Documents\Java>javac exp26_2.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\Bina Computer\OneDrive\Documents\Java>java exp26_2
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Bank transaction started.
Initial Balance = Rs.1000
Deposit Thread deposited Rs.500 | Balance = Rs.1500
Withdrawal Thread withdrew Rs.400 | Balance = Rs.1100
Deposit Thread deposited Rs.700 | Balance = Rs.1800
Withdrawal Thread withdrew Rs.600 | Balance = Rs.1200
Deposit Thread deposited Rs.300 | Balance = Rs.1500
Withdrawal Thread withdrew Rs.500 | Balance = Rs.1000
Final Balance = Rs.1000
Bank transaction completed.

C:\Users\Bina Computer\OneDrive\Documents\Java>
```

Conclusion	The programs successfully demonstrate how to create and execute multiple threads using the Runnable interface. The first program executes two tasks simultaneously, while the second program demonstrates concurrent bank deposit and withdrawal operations with proper synchronization.